



AUDITORÍA WEB:

Soluciones

Grado en Ingeniería Informática de Gestión y Sistemas de Información:

Sistemas de Gestión de Seguridad de Sistemas de Información

Eder Torres Navas

Liviu Deleanu

Jon Requies Ruiz

Alex Isasi Urrutia

Iker Ciordia Elorriaga

Índice

Índice.....	2
Índice de ilustraciones	3
Herramientas utilizadas	4
ZAP:.....	4
Vulnerabilidades arregladas	5
1. Rotura de control de acceso	5
a. <i>Insecure Direct Object Reference (IDOR)</i>	5
b. <i>Cross-Site Request Forgery (CSRF)</i>	6
2. Fallos criptográficos:	7
a. Almacenamiento de contraseñas.....	7
b. Divulgación de información sensible en URL.....	8
3. Inyección.....	9
a. Inyección SQL general	9
b. Inyección HTML y <i>Cross-Site Scripting (XSS)</i>	10
4. Diseño inseguro	11
5. Configuración de seguridad insuficiente	11
a. Cabecera <i>CSP</i> no configurada	11
b. Cabecera <i>Anti-ClickJacking</i> no configurada.....	14
c. Divulgacion de informacion mediante campo <i>X-Powered-By</i>	15
d. Filtración de información de versión mediante el campo <i>Server</i>	15
e. Falta de encabezado <i>X-Content-Type-Options</i>	16
6. Componentes vulnerables y obsoletos	16
7. Fallos de identificación y autenticación.....	18
8. Fallos en la integridad de datos y software.....	18
9. Fallos en la monitorización de la seguridad.....	18
Bibliografía	20

Índice de ilustraciones

Ilustración 1: Escaneo automático de ZAP	4
Ilustración 2: Uso de variable de sesión 1	5
Ilustración 3: Uso de variable de sesión 2	5
Ilustración 4: Método Post register	8
Ilustración 5: Método Post login.....	8
Ilustración 6: Comparación entre el código antiguo y el nuevo	10
Ilustración 7: Muestra de la no ejecución de etiquetas HTML	10
Ilustración 8: Inserción de cabecera CSP en DockerFile.....	12
Ilustración 9: Código usando script de botones en el html	13
Ilustración 10: Código usando script de botones en el JavaScript.....	13
Ilustración 11: Código de escucha de eventos de botones con addEventListener	14
Ilustración 12: Inserción de cabecera Anti-ClickJacking en el DockerFile.....	15
Ilustración 13: Eliminación de cabecera X-Powered-By en el DockerFile	15
Ilustración 14: Ocultación de información de la cabecera Server en el DockerFile.....	15
Ilustración 15: Inserción de cabecera X-Content-Type-Options en el DockerFile.....	16
Ilustración 16: docker-compose.yml con las nuevas versiones	16
Ilustración 17: Vulnerabilidad asociada a la imagen de MariaDB	17
Ilustración 18: Tabla de usuario con nuevas columnas	18
Ilustración 19: Usuario registrado en el log por realizar 5 intentos fallidos	19

Herramientas utilizadas

ZAP:

Para la realización del segundo estudio de seguridad de nuestra página web utilizamos **OWASP ZAP**, ya que fue la herramienta propuesta en clase. Esta herramienta de código abierto permite realizar ataques automatizados a una página web sin necesidad de tener una gran experiencia en el ámbito de la seguridad informática.

Además, ZAP puede utilizarse de forma manual, lo que la convierte en una opción válida también para usuarios más avanzados.

Tras ejecutar ZAP sobre nuestra página web una vez arreglados los agujeros de seguridad que se nos mostraban en la primera auditoría, obtenemos el siguiente listado de alertas:

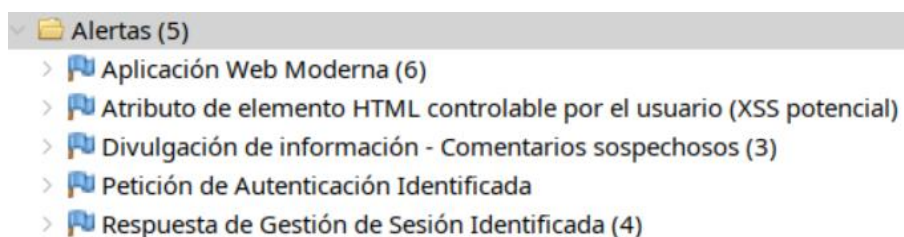


Ilustración 1: Escaneo automático de ZAP

Ninguna de las alertas mostradas actualmente por *ZAP* es un agujero de seguridad, simplemente son falsos positivos o notas informativas. Esto es, hemos arreglado todos los problemas que se indicaban en la primera auditoría.

La primera nos indica que nuestra web está usando técnicas de modernas (*JavaScript dinámico*, *fetch...*).

La segunda es un falso positivo acerca de un posible ataque *XSS* ([apartado 3b](#)) porque en el código del archivo *register.php* donde se genera el token *CSRF* lo detecta como si fuera un campo introducido por el usuario que más adelante se imprime.

La tercera también es una falsa alarma sobre comentarios sospechosos porque *ZAP* no interpreta bien el español. La cuarta simplemente es una alerta informativa sobre haber encontrado la página de identificación.

La quinta también es una alerta informativa sobre haber encontrado información acerca de la identificación una vez *ZAP* ha intentado identificarse.

Vulnerabilidades arregladas

1. Rotura de control de acceso

a. *Insecure Direct Object Reference (IDOR)*

Para solucionar esta brecha de seguridad hemos tenido que añadir cierta lógica de verificación de usuario en el backend *PHP*, es decir, en el servidor. Concretamente, cada vez que un usuario inicie sesión el servidor almacenará su identificador en la variable superglobal `$_SESSION`. Esta variable estará disponible mientras el usuario se encuentre navegando por la web.

Una vez se ha almacenado el identificador que tiene el usuario, en todas las páginas donde se pudiera modificar el identificador para cargar o modificar la información de otro usuario ahora se obviara la información enviada al servidor. Es decir, si el usuario modifica el identificador para hacerse pasar por otro usuario, nunca se le mostrará información o se le permitirá modificar información de ese otro usuario.

Esto es posible porque internamente, en la lógica del servidor donde se utilizaba el identificador proporcionado por el usuario para hacer la consulta a la base de datos, ahora se utiliza el identificador almacenado en la variable `$_SESSION` de tal manera que da igual que el propio usuario modifique el identificador de la *URL* o se haga pasar por otro usuario modificándolo el.

```
//Verificar contraseña
if (password_verify($password, $row['contrasena'])) { // <-- VERIFICAR HASH
    $_SESSION['user_id'] = $row['id']; //Guardar ID de usuario en sesión
    $_SESSION['logged_in'] = true; //Marcar como usuario autenticado
```

Ilustración 2: Uso de variable de sesión 1

```
public function showUserData($user)
{
    session_start(); //Iniciar sesión

    if (!isset($_SESSION['user_id'])) { //Verificar si el usuario está autenticado
        header("HTTP/1.1 401 Unauthorized"); //Código de estado 401 porque no está autenticado
        echo json_encode(['error' => 'No autenticado']);
        return;
    }

    $user_id seguro = $_SESSION['user_id'];

    $conn = new mysqli("db", "admin", "test", "database");
    if ($conn->connect_error)
        die("Database connection failed: " . $conn->connect_error);

    $stmt = $conn->prepare("SELECT * FROM usuarios WHERE id = ?");
    $stmt->bind_param("i", $user_id seguro);
    $stmt->execute();
```

Ilustración 3: Uso de variable de sesión 2

b. Cross-Site Request Forgery (CSRF)

La vulnerabilidad detectada por *ZAP* era que cualquier usuario podía forzar a otro usuario a ejecutar acciones sin su consentimiento, ya que las peticiones POST no contaban con ningún mecanismo de verificación de autenticidad.

La solución a esta vulnerabilidad que presentaba nuestra web consiste en la generación, almacenamiento y validación de un token *CSRF* por sesión, de manera que no se pueda suplantar la identidad de otro usuario.

1. Generación del token

Cada vez que se accede a las vistas *showRegister()* o *showLogin()*, si no existe ningún token, el servidor genera uno aleatorio mediante la función *random_bytes()*:

```
if (empty($_SESSION['csrf_token'])) {  
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
}
```

El token se almacena en la variable de sesión *\$_SESSION['csrf_token']* y se inyecta dentro de los formularios HTML mediante un campo oculto *<input type="hidden">*.

2. Envío del token al servidor

Al enviar el formulario, el token viaja junto con el resto de los datos del usuario ya que es un input como el campo del correo o de la contraseña. El campo oculto dentro de los formularios de registro y de inicio de sesión es el siguiente:

```
<input type="hidden" name="csrf_token" value="<?php echo  
htmlspecialchars($csrf_token, ENT_QUOTES, 'UTF-8'); ?>">
```

3. Validación del token en el servidor

En los métodos *processRegisterForm()* y *processLoginForm()* se comprueban 3 cosas: La primera, que el token exista en la petición, la segunda, que su formato sea válido y la tercera, que coincida con el token almacenado en la sesión del usuario.

```
if (!hash_equals($_SESSION['csrf_token'], $token)) {  
    header('HTTP/1.1 403 Forbidden');  
    echo json_encode(['success' => false, 'message' => 'CSRF token inválido o ausente']);  
    exit;  
}
```

4. Regeneración del token tras acciones sensibles

Al realizarse un inicio de sesión o registro exitoso, el token CSRF se regenera para evitar reutilizaciones.

Gracias a estos 4 pasos, cada formulario queda protegido frente a ataques CSRF, ya que ningún atacante puede conocer ni reutilizar el token de otro usuario.

Durante la implementación de los tokens fue necesario cambiar las vistas de login.html y register.html de ficheros HTML a ficheros PHP. Esto se debe a que los tokens CSRF se generan dinámicamente en el servidor y deben insertarse en los formularios en el momento de su carga. Si las vistas permanecieran en HTML, no habría forma de incluir el valor del token desde el servidor, haciendo imposible validar correctamente las peticiones.

El cambio a PHP permite que el controlador UserController.php incluya directamente las vistas mediante:

```
include __DIR__ . '/../views/login.php';
```

y dentro de ellas se puede usar *PHP* (*<?php echo \$csrf_token; ?>*) para renderizar el valor del token en el formulario.

2. Fallos criptográficos:

a. Almacenamiento de contraseñas

La solución para esta vulnerabilidad es bastante sencilla, puesto que PHP cuenta desde la versión 5.5 con un método nativo para poder “hashear” las contraseñas de manera segura. Por un lado, tendremos que guardar el hash en la base de datos a la hora de hacer un registro. Con la siguiente línea de código ya es suficiente para conseguir lo dicho, antes de guardar el usuario en la base de datos:

```
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
```

Por otro lado, al intentar iniciar sesión, es necesario comparar la contraseña introducida por el usuario con el hash almacenado en la base de datos.

PHP ofrece una función nativa que permite realizar esta verificación de forma segura.

Si la contraseña ingresada coincide con el hash guardado, el sistema permitirá el inicio de sesión correctamente. Esta es la línea de código que se encarga de verificar el hash:

```
password_verify($password, $row['contrasena']);
```

b. Divulgación de información sensible en URL

En la anterior auditoría se detalló que esta vulnerabilidad sucedía a la hora tanto de registrar un usuario nuevo como de iniciar sesión con un usuario existente. ZAP detectó que cuando se realizaban los dos formularios la información del registro y del inicio de sesión se mandaban mediante el método GET, esto hacía que en la *query string* de la URL aparecieran los campos de email y contraseña, por ejemplo.

La solución a esta vulnerabilidad ha sido bastante sencilla, únicamente se ha tenido que cambiar el método de envío de datos del formulario a POST. En nuestro caso no había ningún método especificado y por defecto se utilizaba el método GET. Por lo tanto, incluyendo las siguientes opciones al código HTML de los formularios de registro e inicio de sesión se ha corregido el error:

```
<form id="register_form" method="post">
```

Ilustración 4: Método Post register

```
<form id="login_form" method="post">
```

Ilustración 5: Método Post login

3. Inyección

a. Inyección SQL general

La solución a este agujero de seguridad en nuestra web se daba principalmente por ejecutar sentencias *SQL* directamente con los parámetros que el usuario introducía. Para solucionarlo, basta con sustituir la lógica de ejecución de las sentencias a sentencias *SQL* preparadas (*Prepared Statement*).

En esta forma de ejecutar sentencias *SQL* el primer paso es preparar cuál va a ser el tipo de sentencia que se va a ejecutar almacenándola en una variable que por consenso se suele llamar *stmt*. Usaremos de ejemplo la comprobación de la existencia de un usuario en la base de datos como ejemplo:

```
$stmt = $conn->prepare("SELECT U.DNI FROM usuarios AS U WHERE U.DNI = ?");
```

Ahora se indica el tipo de parámetro que es el parámetro que se indica con el signo de interrogación y de donde va a proceder:

```
$stmt->bind_param("s", $dni);
```

En nuestro caso procede es de tipo *string* y procede de la variable que almacena el DNI. Luego ejecutamos la sentencia *SQL*:

```
$stmt->execute();
```

Almacenamos el resultado de la instrucción en una variable por se trata de una consulta de información (si fuera una inserción, borrado, actualización... no haría falta) y hacemos las comprobaciones necesarias con la misma.

```
$resultado = $stmt->get_result();
```

Estos pasos lo que hacen es preparar la sentencia que se va a ejecutar, para que la base de datos ya sepa qué es lo que se quiere obtener, pero sin indicarle la información externa que nosotros tenemos el usuario tiene que pasarle. De esta forma, cuando se le pasa dicha información a la base de datos, esta ya sabe que no puede interpretarla como parte de la consulta porque de forma previa se ha definido cuál es. Así conseguimos evitar la inyección *SQL*.

Cabe destacar que no es necesario utilizar sentencias preparadas en aquellas consultas que no requieran de parámetros externos al propio sistema porque no se genera ningún agujero de seguridad relacionado con la inyección *SQL*.

b. Inyección HTML y Cross-Site Scripting (XSS)

La mitigación principal para esta vulnerabilidad es escapar la salida antes de cualquier renderización de los datos controlados por el usuario. En nuestro caso, la aplicación utilizaba *innerHTML* para insertar contenido dinámico, lo que permitía interpretar código inyectado. Como solución, proponemos el uso de *textContent* en lugar de *innerHTML*, de forma que el navegador trate dichos datos como texto plano, evitando así, que cualquier *HTML* o *JavaScript* sea ejecutado dentro de los nodos del DOM

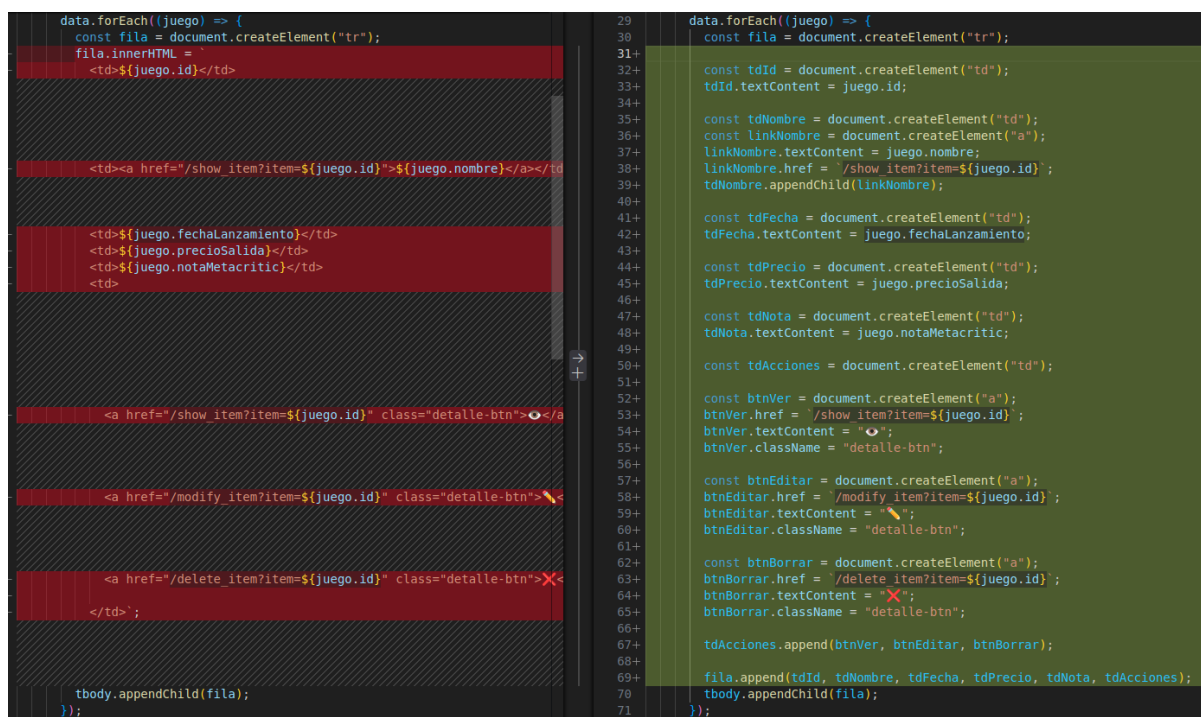


Ilustración 6: Comparación entre el código antiguo y el nuevo

Como resultado, cualquier contenido malicioso se interpreta literalmente y evita la ejecución de cualquier script.

ID	Nombre	Fecha de Lanzamiento	Precio Salida	Puntuacion en Metacritic
2	Grand Theft Auto 5	2013-09-17	49.99	8.50
7	WarHammer	1212-12-21	12.00	9.00
12	 Juego Malicioso 	1212-12-21	19.95	9.00

✚ Añadir juego

Ilustración 7: Muestra de la no ejecución de etiquetas HTML

Además, se ha hecho una revisión completa del sistema, asegurando que cada creación de nodos del DOM aplique la misma estrategia de protección.

Aunque esta medida cubre el escenario principal, como protección en profundidad se recomienda: validación y escape en el servidor, sanitización de entradas y correcta configuración de la cabecera *CSP* (*Content Security Policy*).

4. Diseño inseguro

Para corregir esta vulnerabilidad verificamos que el código reutilizado no contenga vulnerabilidades y que su funcionamiento sea seguro y coherente en todos los módulos donde se implementa. Para ello probamos el código original en distintos casos y comprobamos que no contiene fallas. Además, cada vez que reutilizamos el código nos aseguramos de copiarlo y pegarlo correctamente sin solapar ni meter líneas de por medio.

Es por ello por lo que como mencionamos en el anterior documento la reutilización controlada de código también representa una buena práctica de desarrollo, ya que favorece la consistencia, acelera el proceso de implementación y reduce la probabilidad de introducir errores por reescritura manual.

5. Configuración de seguridad insuficiente

a. Cabecera *CSP* no configurada

La solución al agujero de seguridad sobre no tener configurada la cabecera *CSP* se hace principalmente indicando en el *DockerFile* que en el archivo de configuración del *VirtualHost* se permitan añadir cabeceras *http* e indicar que toda respuesta enviada por el servidor incluya dicha cabecera. Para ello añadimos las siguientes líneas en el *DockerFile*:

```
FROM php:8.4-apache

# Habilitar módulos esenciales
RUN a2enmod rewrite
RUN a2enmod mpm_prefork # Asegurar que usamos prefork para PHP
RUN a2enmod headers #Habilitar el módulo que permite añadir cabeceras HTTP

# Instalar extensiones PHP necesarias
RUN docker-php-ext-install mysqli pdo pdo_mysql

# Configurar Apache para procesar PHP
RUN echo "<VirtualHost *:80>" > /etc/apache2/sites-available/000-default.conf
RUN echo "    ServerName localhost" >> /etc/apache2/sites-available/000-default.conf
RUN echo "    DocumentRoot /var/www/html" >> /etc/apache2/sites-available/000-default.conf
RUN echo "    <Directory /var/www/html>" >> /etc/apache2/sites-available/000-default.conf
RUN echo "        Header always set Content-Security-Policy \"default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'self';\" >> /etc/apache2/sites-available/000-default.conf
RUN echo "        Options -Indexes +FollowSymLinks" >> /etc/apache2/sites-available/000-default.conf
RUN echo "        AllowOverride All" >> /etc/apache2/sites-available/000-default.conf
```

Ilustración 8: Inserción de cabecera CSP en *DockerFile*

La primera habilita el poder añadir cabeceras *http* en el *VirtualHost*, la segunda indica que toda respuesta que devuelva el servidor va a incluir la cabecera *CSP* junto con las directivas acerca de los directorios desde los que se pueden cargar los recursos.

Sin embargo, cabe destacar que ciertas decisiones de diseño en el código han implicado que incluir la cabecera sea más difícil que simplemente esto. Inicialmente el funcionamiento de hacer “click” en la mayoría de los botones en la web se implementaba directamente en el *html*, pero añadir la cabecera *CSP* ha hecho que estos dejen de funcionar porque no permitía cargar sus *scripts*, con el fin de evitar los ataques *XSS*. Para poder solucionarlo se ha tenido que mover la funcionalidad del “click” a los ficheros *JavaScript* correspondientes utilizando *eventListeners*.

```

<form id="register_form" onsubmit="submitForm(this); return false;">
  <label>Nombre y apellidos:</label>
  <input name="nombreapellido" type="text" placeholder="Nombre y apellidos">

  <label>DNI:</label>
  <input name="DNI" type="text" placeholder="12345678-X">

  <label>Contraseña:</label>
  <input name="password" type="password" placeholder="Contraseña segura">

  <label>Teléfono:</label>
  <input name="telefono" type="text" placeholder="685123456">

  <label>Fecha de Nacimiento:</label>
  <input name="fechanac" type="text" placeholder="aaaa-mm-dd">

  <label>Email:</label>
  <input name="email" type="text" placeholder="tuemail@dominio.com">

  <button id="register_submit" type="submit">  Registrarse</button>

```

Ilustración 9: Código usando *script* de botones en el *html*

```

<form id="register_form">
  <label>Nombre y apellidos:</label>
  <input name="nombreapellido" type="text" placeholder="Nombre y apellidos">

  <label>DNI:</label>
  <input name="DNI" type="text" placeholder="12345678-X">

  <label>Contraseña:</label>
  <input name="password" type="password" placeholder="Contraseña segura">

  <label>Teléfono:</label>
  <input name="telefono" type="text" placeholder="685123456">

  <label>Fecha de Nacimiento:</label>
  <input name="fechanac" type="text" placeholder="aaaa-mm-dd">

  <label>Email:</label>
  <input name="email" type="text" placeholder="tuemail@dominio.com">

  <button id="register_submit" type="submit">  Registrarse</button>

```

Ilustración 10: Código usando *script* de botones en el *JavaScript*

```

document.addEventListener("DOMContentLoaded", () => {
    window.createProfileCircle();

    //Buscar el formulario en la página
    const registerForm = document.getElementById("register_form");

    //Asegurarse de que el formulario exista en esta página
    if (registerForm) {

        //Añadir el "escuchador" para el evento 'submit'
        registerForm.addEventListener("submit", (event) => {
            event.preventDefault();

            submitForm(event.target);
        });
    }
});

```

Ilustración 11: Código de escucha de eventos de botones con *addEventListener*

b. Cabecera *Anti-ClickJacking* no configurada

La solución a este agujero de seguridad es idéntica a la anterior. Es decir, con la configuración de la cabecera *CSP* hemos también conseguido evitar posibles ataques de *clickjacking*. Es decir, hasta 2015 era necesario configurar la cabecera *X-Frame-Options* para evitar este tipo de ataques. A partir de entonces, se lanzó una revisión de la cabecera *CSP* que también solucionaba este agujero. Para ello simplemente había que incluir la política “frame-ancestors ‘self’” al resto de políticas.

De igual manera, si quisiéramos configurar de forma independiente la cabecera *X-Frame-Options*, simplemente tendríamos que añadir la siguiente línea en nuestro *DockerFile*, habilitando así la cabecera específica.

```

RUN echo " <Directory /var/www/html>" >> /etc/apache2/sites-available/000-default.conf
RUN echo " Header always set Content-Security-Policy \"default-src 'self'; script-src 'self'; object-src 'none';
base-uri 'self'; form-action 'self'; frame-ancestors 'self';\" >> /etc/apache2/sites-available/000-default.conf
7 RUN echo " Header always set X-Frame-Options \"SAMEORIGIN\" >> /etc/apache2/sites-available/000-default.conf
RUN echo " Options -Indexes +FollowSymLinks" >> /etc/apache2/sites-available/000-default.conf
RUN echo " AllowOverride All" >> /etc/apache2/sites-available/000-default.conf
RUN echo " Require all granted" >> /etc/apache2/sites-available/000-default.conf

```

Ilustración 12: Inserción de cabecera *Anti-ClickJacking* en el *DockerFile*

Esta indica que queremos que nuestra web solamente pueda cargar *iframes* que tengan como origen el mismo que tiene la propia web. Con esto evitamos que un atacante pueda insertar un *iframe* malicioso externo.

c. Divulgacion de informacion mediante campo *X-Powered-By*

Este agujero de seguridad se soluciona de la misma manera que las 2 anteriores, pero indicándole al *VirtualHost* que cada vez que vaya a enviar una respuesta al cliente, la intercepte y elimine la cabecera *X-Powered-By*. Para ello solo añadimos la siguiente línea en el *DockerFile*.

```

RUN echo " Header always set Content-Security-Policy \"default-src 'self'; script-src 'self'; object-src 'none';
base-uri 'self'; form-action 'self'; frame-ancestors 'self';\" >> /etc/apache2/sites-available/000-default.conf
RUN echo " Header always set X-Frame-Options \"SAMEORIGIN\" >> /etc/apache2/sites-available/000-default.conf
RUN echo " Header unset X-Powered-By" >> /etc/apache2/sites-available/000-default.conf
RUN echo " Options -Indexes +FollowSymLinks" >> /etc/apache2/sites-available/000-default.conf

```

Ilustración 13: Eliminación de cabecera *X-Powered-By* en el *DockerFile*

d. Filtración de información de versión mediante el campo *Server*

La solución a este agujero de seguridad es muy parecida a los anteriores. En este caso, no añadimos una directiva que le indique a *Apache* que elimine dicha cabecera, sino que le decimos que oculte la información relacionada con el sistema operativo del servidor. Para ello incluimos las siguientes 2 líneas en nuestro *Docker File*:

```

información del SO
RUN sed -i 's/^[ \t]*ServerTokens OS[ \t]*$/ServerTokens Prod/' /etc/apache2/conf-enabled/security.conf
# Reemplaza la línea "ServerSignature On" por "ServerSignature Off" para ocultar la información de páginas de
error
RUN sed -i 's/^[ \t]*ServerSignature On[ \t]*$/ServerSignature Off/' /etc/apache2/conf-enabled/security.conf

```

Ilustración 14: Ocultación de información de la cabecera *Server* en el *DockerFile*

La primera línea lo que hace es buscar en el fichero *security.conf* de *Apache* la línea *ServerTokens OS* y la sustituye por *ServerTokens Prod*. Con esto conseguimos que *Apache* pase de mostrar toda la información acerca del sistema operativo del servidor a no mostrarla.

Con la segunda línea hacemos que se sustituya la línea *ServerSignature On* por *ServerSignature Off*. Con esto evitamos que en caso de que el usuario consiga acceder a una página de error esta no muestre tampoco dicha información.

e. Falta de encabezado *X-Content-Type-Options*

Este agujero de seguridad se soluciona de la misma manera que los anteriores. Simplemente tenemos que añadir una línea en el *DockerFile* incluyendo la cláusula de tal forma que se habilite el envío de la cabecera *X-Content-Type-Options* en todas las respuestas que envíe el servidor.

```
RUN echo "  Header always set X-Frame-Options \"SAMEORIGIN\"" >> /etc/apache2/sites-available/000-default.conf
RUN echo "  Header unset X-Powered-By" >> /etc/apache2/sites-available/000-default.conf
RUN echo "  Header always set X-Content-Type-Options \"nosniff\"" >> /etc/apache2/sites-available/000-default.conf
RUN echo "  Options -Indexes +FollowSymLinks" >> /etc/apache2/sites-available/000-default.conf
```

Ilustración 15: Inserción de cabecera *X-Content-Type-Options* en el *DockerFile*

6. Componentes vulnerables y obsoletos

Para resolver la vulnerabilidad encontrada en la primera auditoría simplemente tenemos que especificar las versiones de las imágenes que queremos utilizar, investigando previamente si dichas versiones son seguras. Además, tenemos que revisar periódicamente las nuevas versiones que se vayan lanzando para ir actualizando las tecnologías usadas en nuestra página web.

Anteriormente usábamos las versiones 10.8.2 de *MariaDB*, 8.1 de *PHP* y 5.2.2 de *phpMyAdmin*. Actualmente hemos actualizado las versiones a las estables más recientes: 12.0.2 de *MariaDB*, 8.4.14 de *PHP* y 5.2.3 de *phpMyAdmin*.

```
db:
  image: mariadb:12.0.2
  restart: always
  volumes:
    - ./mysql:/var/lib/mysql

phpmyadmin:
  image: phpmyadmin/phpmyadmin:5.2.3
  links:
    - db
  ports:
    - 8890:80

FROM php:8.4-apache
```

Ilustración 16: *docker-compose.yml* con las nuevas versiones

En nuestro caso, como utilizamos imágenes de empresas reconocidas a nivel mundial podemos confiar en que sus imágenes son seguras. Si utilizáramos imágenes de Docker provenientes de fuentes desconocidas, sería recomendable analizarlas con distintas herramientas, como *Docker Scout*.

Para analizar con Docker Scout (suponiendo que ya está instalado) una imagen local propia en formato *.tar* basta con tener la imagen local que se quiere analizar en el directorio actual y ejecutar el siguiente comando:

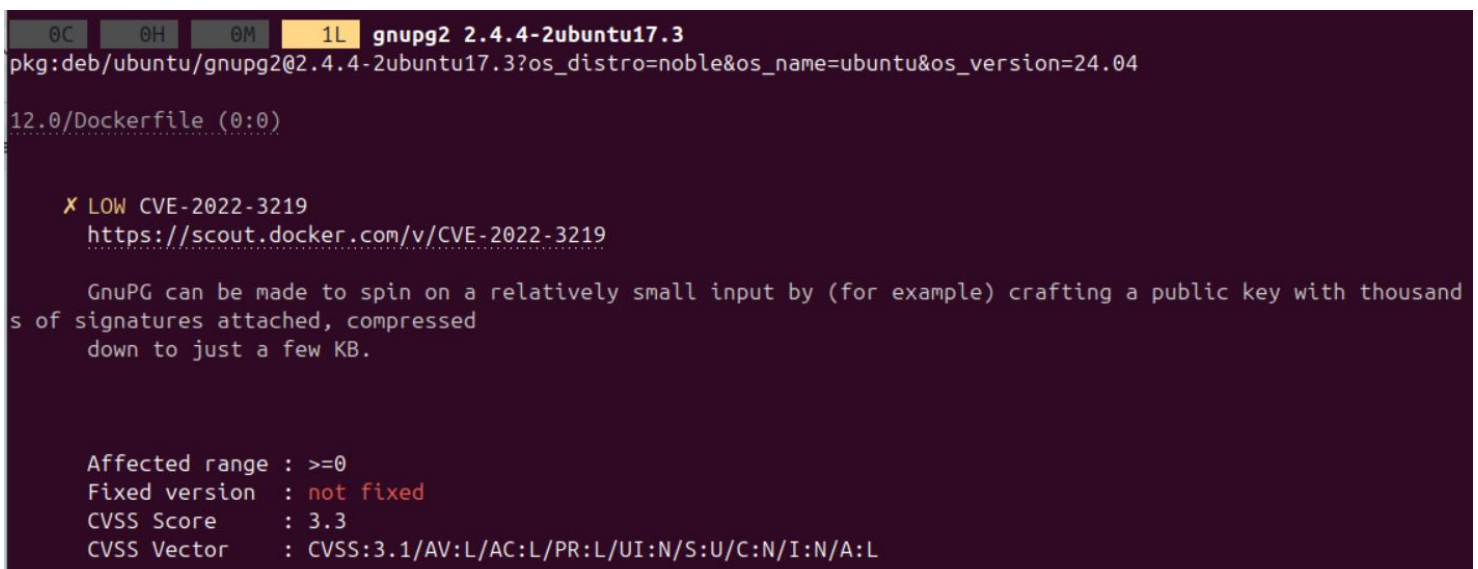
```
docker scout cves archive://nombreimagen.tar --details
```

Esto nos mostrará una lista de todas las vulnerabilidades que haya encontrado *Docker Scout* sobre todos los paquetes que use nuestra imagen.

Si queremos analizar la imagen de un repositorio de *Docker* ejecutamos:

```
docker scout cves nombreimagen:version --details
```

Por ejemplo, para analizar la imagen de *MariaDB* y ver si realmente es segura o no, sustituimos “imagen” por “mariadb” y “version” por “12.0.2” (la estable más reciente hasta el momento de escritura). Actualmente la lista de vulnerabilidades que nos muestra *Docker Scout* son todas sobre paquetes externos que utiliza *MariaDB*, pero no del propio código de *MariaDB*, por lo que ya tenemos algo en lo que basarnos para poder confiar parcialmente de la imagen.



```
0C 0H 0M 1L gnupg2 2.4.4-2ubuntu17.3
pkg:deb/ubuntu/gnupg2@2.4.4-2ubuntu17.3?os_distro=noble&os_name=ubuntu&os_version=24.04
12.0/Dockerfile (0:0)

X LOW CVE-2022-3219
https://scout.docker.com/v/CVE-2022-3219

GnuPG can be made to spin on a relatively small input by (for example) crafting a public key with thousand
s of signatures attached, compressed
down to just a few KB.

Affected range : >=0
Fixed version  : not fixed
CVSS Score     : 3.3
CVSS Vector    : CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L
```

Ilustración 17: Vulnerabilidad asociada a la imagen de *MariaDB*

7. Fallos de identificación y autenticación

Para resolver la vulnerabilidad recogida en la anterior auditoría relacionada con los ataques de fuerza bruta se ha implementado una solución que consiste en limitar el número de intentos de inicio de sesión de una cuenta a 5 con un intervalo de espera de 1 minuto, es decir, un usuario podrá intentar iniciar sesión con su cuenta fallando la contraseña hasta 5 veces. Una vez fallado el quinto intento el usuario deberá esperar 1 minuto hasta volver a recibir otros 5 intentos. De esta manera, se consigue que los ataques de fuerza bruta sean más difíciles de realizar ya que el atacante tendrá que esperar.

La implementación utilizada en este caso hace que en la base de datos se registre el número de intentos de inicio de sesión fallidos que lleva la cuenta en particular desde cualquier ventana, es decir, que el atacante no podrá abrir un número grande de ventanas en su navegador y utilizar 5 intentos en cada ventana simultáneamente.

Para realizar esta implementación se han añadido a la tabla usuarios de la base de datos 2 nuevos atributos. El primero cuenta el número de intentos fallidos que lleva esa cuenta antes de reiniciarse tras pasar 1 minuto, y el segundo guarda el momento en el que se falló el quinto intento, para poder calcular el minuto de espera.

id	usuario	nombre	dni	telefono	fechaNacimiento	email	contrasena	login_fallidos	momento_login
----	---------	--------	-----	----------	-----------------	-------	------------	----------------	---------------

Ilustración 18: Tabla de usuario con nuevas columnas

8. Fallos en la integridad de datos y software

En nuestro caso los problemas derivados de la falta de integridad de datos y software ya se han tratado en el resto de puntos o no se dan directamente. Por ejemplo, al no usar librerías externas no tenemos que verificar que dichas librerías realmente procedan de lugares seguros y sean seguras.

Las posibles soluciones a las posibles brechas de seguridad ya se explicaron en la auditoría anterior.

9. Fallos en la monitorización de la seguridad

En este apartado tenemos diversas posibles brechas de seguridad que se pueden tratar de distintas formas. Por ejemplo, en la auditoría se mencionaba que una posible brecha de forma indirecta es el no guardar ningún *log* sobre posibles intentos de inicio de sesión fallidos que nos permitiera conocer si nuestra web estaba sufriendo de algún ataque de fuerza bruta. Para solucionarlo hemos decidido guardar en un fichero a forma de *log* cada vez que un usuario llega al límite de 5 intentos de inicio de sesión sobre una cuenta concreta.

Cada vez que un usuario intente iniciar sesión y falle en cinco ocasiones consecutivas, se registrará la información en un fichero de texto. Dicho fichero contendrá la fecha y la hora del quinto intento fallido, el nombre de usuario de la cuenta a la que se intentó acceder, la dirección IP desde la que se realizó el intento y el motivo por el cual se guarda el registro.

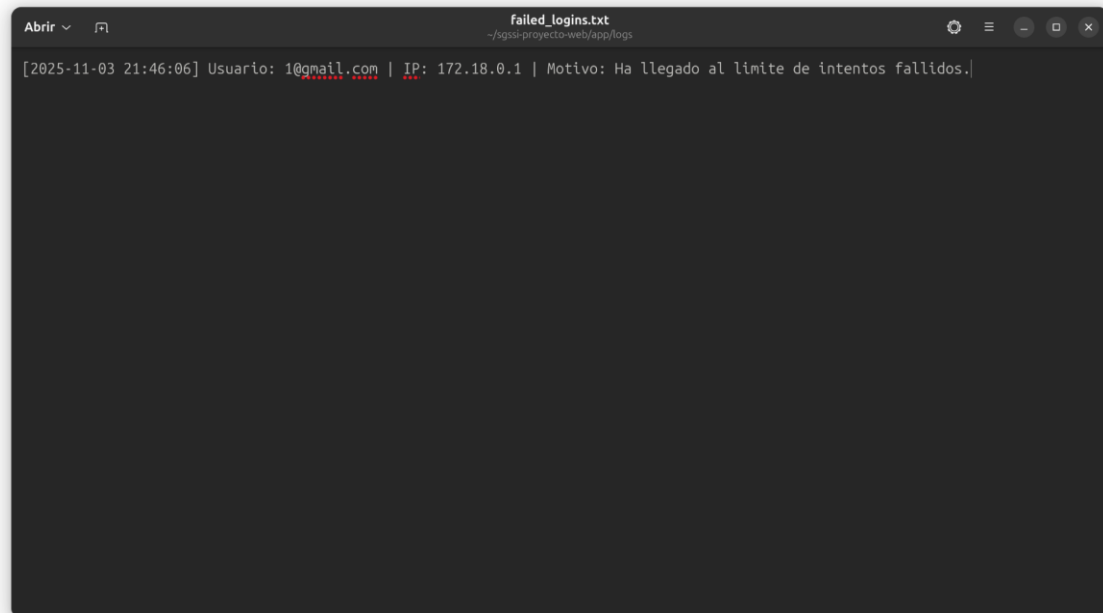


Ilustración 19: Usuario registrado en el log por realizar 5 intentos fallidos

Otra brecha de seguridad sería la de no llevar el control de versiones de las tecnologías mencionado en el [apartado 6](#) que está altamente relacionado con la monitorización de seguridad. Con ello en mente, es una buena práctica hacer revisiones periódicas sobre las tecnologías que usa nuestra web con el objetivo de ir actualizándolas a las más recientes y así arreglar posibles brechas de seguridad que estas pudieran tener.

Bibliografía

Docke. (s.f.). *Docker Scout Image analysis*. Recuperado de <https://docs.docker.com/scout/explore/analysis/>

Webdock. (s.f.). *How to configure security headers in Nginx and Apache*. Recuperado de <https://webdock.io/en/docs/how-guides/security-guides/how-to-configure-security-headers-in-nginx-and-apache>

MDN Web Docs. (s.f.). *X-Frame-Options header*. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>

Secureflag. (s.f.). *Cross-site request forgery (CSRF) in PHP*. Recuperado de https://knowledge-base.secureflag.com/vulnerabilities/cross_site_request_forgery/cross_site_request_forgery_php.html

IONOS. (24.6.2020). *Prepared statements: introducción y ejemplos*. Recuperado de <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/prepared-statements/>